BrainAI

ProSuite Industrial

PROSUITE INDUSTRIAL

PRODUKT-DOKUMENTATION

BrainAI (Inhaber: Sascha Alexander Köhne)

BrainAI Inhaber: Sascha Alexander Köhne	PROSUITE INDUSTRIAL	🟢 ÖFFENTLICHE KLASSIFIZIERUNG
Core Architecture & Development	**Technical Documentation**	Zugriffsstufe: Öffentlich / Beta

---

Revisionshistorie

Version	Datum	Autor	Status	Beschreibung
2.0.0	28.02.2026	S. Köhne	Stable	Initial Release: Modulare CSS Architektur & Adaptive Theme Engine

1. Einleitung: Die ProSuite CSS-Architektur

Die grafische Benutzeroberfläche (GUI) der ProSuite wurde nach dem Prinzip der **Separation of Concerns (SoC)** vollständig modularisiert. Um im ProWB (C-Builder) eine deterministische und fehlerfreie Kompilierung der Stylesheets in den `index.html`-Monolithen zu garantieren, werden die Dateien über ein striktes numerisches Präfix (`00_` bis `99_`) gesteuert.

Dieses System verhindert "CSS Bleeding" (ungewolltes Überschreiben von Styles) und stellt sicher, dass generierte Dokumente (Markdown) und dynamische Krypto-Dashboards perfekt in das jeweilige Corporate Design des Nutzers integriert werden, ohne die Layout-Struktur zu gefährden.

---

2. Modul-Spezifikationen ("Wo ist was?")

Die Architektur ist in fünf funktionale Ebenen unterteilt, die nacheinander in den DOM geladen werden:

Ebene 1: Das Fundament (`00_`)

- **`00_reset.css`:** W3C-Standardisierung. Eliminiert browser-spezifische Eigenheiten (Padding, Margin, List-Styles), um auf allen Endgeräten eine identische Ausgangslage (Zero-Baseline) zu schaffen.
- **`00_vars.css`:** Definiert die globalen CSS-Variablen (`:root`). Dies ist das "Fallback-Theme" (Siemens Industrial Base), auf das das System zurückgreift, falls kein spezifisches Theme geladen ist.

Ebene 2: Das Skelett (`10_`)

- **`10_layout.css`:** Beinhaltet ausschließlich die globale App-Shell. Hier werden Header, Navigation, Footer und der flexible Haupt-Container (`#app-view-container`) gestylt. Es enthält **keine** inhaltsspezifischen Farben, sondern referenziert nur CSS-Variablen.

Ebene 3: Die Dokumenten-Engine (`20_`)

- **`20_markdown.css`:** Das exklusive Styling für den C-basierten GFM-Parser des ProWB. Es übersetzt nackte HTML-Tags (Tabellen, Code-Blöcke, Checkboxen) in das ProSuite-Design. **Sicherheitsmerkmal:** Diese Datei nutzt zu 100 % adaptive CSS-Variablen und besitzt *keine* hartcodierten Farbwerte. Dadurch passen sich Audit-Handbücher automatisch an das ausgewählte Theme an (Light/Dark/Matrix).

Ebene 4: UI & Dashboards (`40_` bis `50_`)

- **`40_dashboard.css`:** Definiert die responsive Kachel-Struktur (Grid), Action-Buttons und das Terminal-Fenster (`.console-window`) für die Hauptansichten.
- **`50_components.css`:** Stellt mikroskopische UI-Elemente wie Eingabefelder (`.std-input`), Hash-Ausgabefenster und Status-LEDs (`.led-big`) bereit.

Ebene 5: Die Theme-Engine (`99_`)

- **`99_theme_*.css`:** Diese Dateien überschreiben gezielt die Variablen aus `00_vars.css`, um das gesamte Aussehen der Suite auf Knopfdruck zu verändern, ohne Layout-Dateien anfassen zu müssen.

3. Die Adaptive Theme-Engine (Integration & Anpassung)

Die ProSuite nutzt ein datengesteuertes Theming-System. Das JavaScript (`30_ui_general.js`) ändert lediglich das Attribut am `<body>`-Tag (z. B. `<body data-theme="matrix">`). Die CSS-Engine greift diesen State auf und injiziert global neue Farbpaletten.

3.1 Notwendige Variablen für ein neues Theme

Um ein neues Theme (z. B. `99_theme_ocean.css`) zu erstellen, **muss** der Block `body[data-theme="ocean"]` folgende Kern-Variablen zwingend überschreiben:

```
body[data-theme="ocean"] {
    /* Hintergrund & Flächen */
    --color-bg: #001f3f;          /* Äußerster Hintergrund */
    --color-surface: #003366;     /* Hintergrund für Kacheln & Tabell
    --color-header-bg: #000a1a;   /* Header, Tabellenköpfe & Footer *
    --color-nav-bg: #002244;      /* Navigationsleiste & Code-Block R

    /* Typographie & Kontrast */
    --text-main: #e0f7fa;         /* Hauptschriftfarbe (Fließtext) */
    --text-muted: #80cadd;        /* Labels und Zitate */
    --text-light: #ffffff;        /* Zwingend kontrastreich für Heade

    /* Signalfarben & Akzente */
    --color-accent: #00e5ff;      /* Primäre Interaktionsfarbe (Butto
    --color-accent-dim: #00b8cc;  /* Hover-Zustände */
    --color-danger: #ff4d4d;      /* Fehler & Panic-Wipe */

    /* Effekte & Typographie */
    --shadow-metallic: 0 0 10px rgba(0, 229, 255, 0.3); /* Neon/Glow-
    --shadow-card: 0 4px 10px rgba(0,0,0,0.5);

    --font-stack: 'Segoe UI', sans-serif;
    --font-mono: 'Courier New', monospace; /* Zwingend für Terminal &
}
```

3.2 Theme-Spezifische Overrides (Component Overrides)

In 95 % der Fälle reicht das Überschreiben der CSS-Variablen aus. Wenn ein Theme jedoch bauliche Anpassungen erfordert (z. B. das Matrix-Theme, das harte Rahmen um Dash-Cards benötigt, die in anderen Themes weiche Schatten sind), muss das Override **strikt innerhalb der Theme-Datei** erfolgen.

Regel: Niemals die `40_dashboard.css` für ein Theme anpassen! **Korrekt:** Das Override in die `99_theme_matrix.css` schreiben:

```
/* Beispiel für legitimes Override innerhalb einer Theme-Datei */
body[data-theme="matrix"] .dash-card {
    border: 1px solid #00ff00; /* Harter Rand nur im Matrix-Modus */
    box-shadow: none;          /* Schatten deaktivieren */
}
```

4. Best Practices für das Styling

1. **Keine Hardcodes in UI/Markdown:** In den Layout- und Markdown-Dateien (`10_` bis `50_`) dürfen **niemals** feste HEX-Werte (z. B. `color: #333`) verwendet werden. Es müssen immer Variablen (z. B. `color: var(--text-main)`) referenziert werden, da andernfalls das Theme-Umschalten an diesen Stellen bricht.
2. **Native HTML-Elemente:** Checkboxen in Task-Listen (ProTU-Audit) werden nicht durch komplexe Custom-Grafiken ersetzt. Wir nutzen stattdessen `accent-color: var(--color-accent);` auf der nativen HTML-Checkbox. Dies garantiert maximale Performance ohne DOM-Overhead.
3. **Scrollbar-Hygiene:** Custom-Scrollbars (wie im Terminal) sollten subtil an den `background` und `color-surface` des jeweiligen Themes angepasst sein, um visuelle Brüche zu vermeiden.

© 2026 Sascha Köhne	✉️ **Contact for KYC:** koehne83@googlemail.com	🛡️ **ProTU-Verified**

System Architect	BrainAI Inhaber : Sascha Alexander Köhne	We don't need **BRUTEFORCE**, we know **PHYSICS**